

Практическая работа «Применение алгоритмов цифровой обработки изображений средствами библиотек компьютерного зрения»

Цель: получение навыков работы с библиотеками компьютерного зрения OpenCV и dlib для базовых задач обработки изображений.

Содержание отчета: ответы на контрольные вопросы, выполненные задания и вывод о проделанной работе.

Теоретическая часть

Обработка изображений, компьютерное зрение, распознавание образов и машинное обучение являются тесно связанными областями и, как правило, при решении сложных задач практически все они реализуются в прикладных системах.

Обработка изображений (в узком смысле) решает задачи преобразования изображений, поэтому исходными и результирующими данными этого этапа являются цифровые изображения. В прикладном плане – это построение некоторой системы (или программного комплекса), которая решает определенный класс задач от получения цифрового изображения до интерпретации его содержания и извлечения некоторой информации.

Компьютерное зрение (Computer (Machine) Vision) занимается анализом образов. Для некоторой сцены (аудитория, комната и др.) компьютер должен дать ее описание (есть ли в ней предметы, какая освещенность и т. д.). Таким образом, компьютерное «зрение» переводит изображение в описание.

Распознавание – это процесс отнесения изображения к некоторому определенному классу. В широком смысле допустимо трактовать распознавание как построение полной модели изображения, которую можно воспроизвести посредством машинной графики.

Изображения представляют собой специфический вид информации и их специфика еще не полностью изучена ни в исследованиях, посвященных изучению зрительного восприятия человека, ни в информатике. Одной из особенностей является избыточность практически любого изображения. Причем изображение одной и той же сцены реального мира несет разную информацию разным пользователям, что не позволяет значительно уменьшить объем данных, содержащихся, например, в космическом снимке, хранящемся в базе данных изображений.

Цифровым изображением называются данные, организованные в виде числового массива, воспроизводящего свойства некоторой реальной сцены или синтезированного человеком объекта (чертежа, карты) и тесно связанного со способом получения этих данных.

С формальной точки зрения цифровое изображение представляется в виде матрицы действительных чисел F с элементами f_{ij} (каждому элементу ставится в соответствие некоторое число – код яркости, обычно от 0 до 255), где ij – целочисленные координаты. Причем принято за начало координат принимать левый верхний угол изображения, где $i=0, j=0$.

С использованием введенных обозначений можно компактно записать полное цифровое изображение размерами $M \times N$ в форме матрицы F :

$$F = \begin{bmatrix} f_{00} & f_{01} & \cdots & f_{0,N-1} \\ f_{10} & f_{11} & \cdots & f_{1,N-1} \\ \cdots & \cdots & \cdots & \cdots \\ f_{M-1,0} & f_{M-1,1} & \cdots & f_{M-1,N-1} \end{bmatrix} = [f_{ij}].$$

Каждый элемент этой матрицы называется элементом изображения или пикселем (*pixel* от английского *picture element*). Каждый пиксель цветного изображения в системах компьютерного зрения описывается тремя уровнями яркости – красным (R), зеленым (G) и синим (B) в диапазоне от 0 до 255.

В качестве примеров систем, использующих информацию из изображений и видео, требующих применения методов обработки изображений, компьютерного зрения и распознавания образов можно привести системы управления процессами (промышленные роботы, автономные транспортные средства); системы видеонаблюдения с автоматизированным или автоматическим анализом видеоданных; системы организации информации (например, индексация баз данных изображений); системы моделирования объектов или окружающей среды (анализ медицинских изображений, топографическое моделирование); системы человеко-машинного взаимодействия; системы дополненной реальности и др.

Множество методов обработки изображений делится на методы обработки в частотной и пространственной областях. Термин *пространственная область* относится к плоскости изображения как таковой, и данная категория объединяет подходы, основанные на прямом манипулировании пикселями изображения. Способы обработки изображений в *частотной*

области (после применения дискретного преобразования Фурье) достаточно развиты, но требуют значительно больших вычислительных затрат и для решения практических задач применяются реже.

В общем виде процессы пространственной обработки изображений описываются выражением

$$g_{ij} = T[f_{ij}],$$

где f_{ij} – элементы входного изображения F ;

g_{ij} – элементы обработанного изображения G ;

T – оператор преобразования входного изображения в окрестности точки с координатами (ij) .

Под окрестностью вокруг точки понимают квадратную или прямоугольную область изображения, которая центрирована в точке с координатами (ij) . Простейшая форма оператора достигается в случае, когда окрестность имеет размеры 1×1 . При этом значение g зависит только от значения f в точке (ij) и оператор T становится функцией градационного преобразования (функцией преобразования интенсивностей или функцией отображения). Примерами являются преобразование цветного изображения в полутоновое, бинаризация, получение негативного изображения, логарифмическое и степенное преобразования и др.).

Преобразование цветного изображения в полутоновое заключается в получении яркости каждой точки по формуле

$$Y = 0,3R + 0,59G + 0,11B,$$

где R, G, B – значение красного, зеленого и синего цветов в обрабатываемой точке, и последующем копировании на все три канала полученной величины.

Бинаризация – это преобразование полутонового изображения к одноцветному (монохромному или бинарному).

Пусть f_{ij} – полутоновое изображение, t – порог и b_0, b_1 – два бинарных значения (для бинарного черно-белого $b_0 = 0, b_1 = 255$). Результат порогового деления – бинарное изображение g_{ij} , полученное следующим образом:

$$g_{ij} = \begin{cases} b_0, & \text{if } f_{ij} \leq t; \\ b_1, & \text{if } f_{ij} > t. \end{cases}$$

Основной задачей является выбор значения t с помощью некоторого критерия. Это значение может выбираться как одинаковым для всего изображения, так и различным для различных его частей. Если значения объектов и фона режима достаточно однородны по всему изображению, то может использоваться одно пороговое значение для всего изображения. Использование единственного значения порога для всех пикселей изображения называется *глобальным пороговым разделением*. Методика *локального порогового разделения* предполагает определение порога для области изображения или пикселей. Такие алгоритмы называются адаптивными алгоритмами бинаризации, к ним можно отнести метод Оцу, Брэдли и др.

Логарифмическое преобразование может быть использовано для изменения яркости и определяется как

$$g_{ij} = c \log(1 + f_{ij}),$$

где c – константа, $f_{ij} \in 0, \dots, L-1$.

Использование логарифма позволяет узкий диапазон малых значений яркости преобразовать в более широкий диапазон выходных значений. Для больших значений входного сигнала верно противоположное утверждение. Такой тип преобразования используется для растяжения диапазона значений темных пикселей на изображении с одновременным сжатием диапазона значений ярких пикселей.

Слабый контраст – один из наиболее распространенных дефектов фотографических и сканированных изображений, обусловленный ограниченностью диапазона воспроизводимых яркостей. Путем цифровой обработки контраст можно повысить, изменяя яркость каждого элемента изображения и увеличивая диапазон яркостей.

Традиционно для цифрового представления каждого отсчета изображения в компьютере отводится 1 байт запоминающего устройства. Следовательно, входной или выходной сигналы могут принимать одно из 256 значений в диапазоне 0–255. Предположим, что минимальная и максимальная яркости исходного изображения равны $f_{\min}$ и $f_{\max}$ соответственно. Если эти параметры или один из них существенно отличаются от граничных значений яркостного диапазона, то изображение выглядит как неконтрастное.

В реальных изображениях обычно существует перекося в сторону малых уровней и яркость большинства элементов изображения ниже средней. На темных участках подобных изображений детали часто оказываются

неразличимыми. Одним из методов улучшения таких изображений является видоизменение гистограммы. Этот метод предусматривает преобразование яркостей исходного изображения с тем, чтобы гистограмма распределения яркостей обработанного изображения приняла желаемую форму.

Одним из методов улучшения контраста является линейная растяжка гистограммы, когда уровням исходного изображения, лежащим в интервале $[f_{\min}, f_{\max}]$, присваиваются новые значения с тем, чтобы охватить весь возможный интервал изменения яркости, в данном случае $[0, 255]$. При линейном контрастировании используется линейное поэлементное преобразование вида

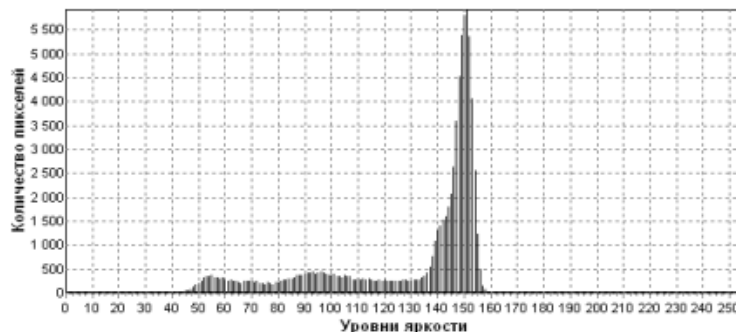
$$g_{ij} = \frac{f_{ij} - f_{\min}}{f_{\max} - f_{\min}}(g_{\max} - g_{\min}) + g_{\min}.$$

Такой алгоритм заложен во многих программных продуктах, реализующих функцию «Autocontrast».

При нормализации гистограммы на весь максимальный интервал уровней яркости $[0, 255]$ растягивается не вся гистограмма, лежащая в пределах $[f_{\min}, f_{\max}]$, а ее наиболее интенсивный участок $[f'_{\min}, f'_{\max}]$, из рассмотрения исключаются малоинформативные начальный и конечный участки. Например, для рассматриваемого изображения на основе анализа гистограммы исходного изображения примем $f'_{\min} = 45$ и $f'_{\max} = 160$.

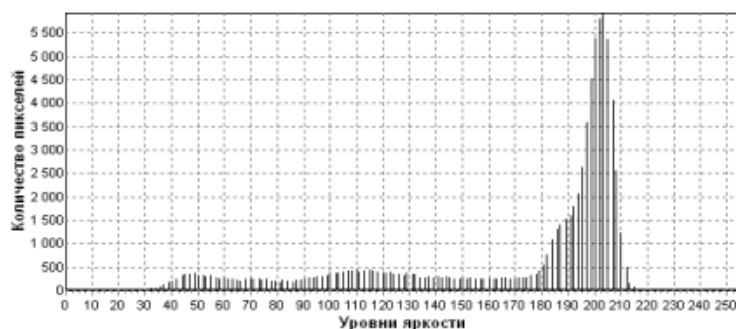
Целью выравнивания гистограммы (линеаризации, эквализации) является такое преобразование, чтобы в идеале все уровни яркости приобрели бы одинаковую частоту, а гистограмма яркостей отвечала бы равномерному закону распределения. Кроме этого, существует метод видоизменения гистограмм, который обеспечивает экспоненциальную или гиперболическую форму распределения яркостей улучшенного изображения. Применяются также «локальные» методы улучшения на основе обработки части изображения (рисунок 1.1).

Другой подход заключается в применении методов, в результате которых на выходное значение пикселя оказывают влияние текущий пиксель и его соседи, т. е. увеличивается окрестность вокруг пикселя и это приводит к значительно большей гибкости при обработке изображений. При этом используется маска (апертура, окно, ядро, окрестность), которая представляет собой двумерный массив заданного размера, чаще всего размером 3×3 . Такие методы называют *обработкой* или *фильтрацией на основе маски*.



а

б



в

г

а – исходное изображение; *б* – гистограмма исходного изображения;
в – результат улучшения контраста; *г* – гистограмма после линейной растяжки

Рисунок 1.1. – Результат улучшения контраста

В качестве маски используется множество весовых коэффициентов, заданных во всех точках окрестности, обычно симметрично окружающих рабочую точку кадра. Распространенным видом окрестности, часто применяемым на практике, является квадрат 3×3 с рабочим элементом в центре (рисунок 1.2). На рисунке 1.3 представлены элементы области и изображения под маской. При цифровой обработке изображений используется декартова система координат с началом в левом верхнем углу кадра и с положительными направлениями из этой точки вниз и вправо.

$W_{(-1,-1)}$	$W_{(-1,0)}$	$W_{(-1,1)}$
$W_{(0,-1)}$	$W_{(0,0)}$	$W_{(0,1)}$
$W_{(1,-1)}$	$W_{(1,0)}$	$W_{(1,1)}$

Рисунок 1.2. – Коэффициенты маски с относительными значениями координат

$f_{(i-1,j-1)}$	$f_{(i-1,j)}$	$f_{(i-1,j+1)}$
$f_{(i,j-1)}$	$f_{(i,j)}$	$f_{(i,j+1)}$
$f_{(i+1,j-1)}$	$f_{(i+1,j)}$	$f_{(i+1,j+1)}$

Рисунок 1.3. – Элементы области изображения под маской

Сущность процедуры фильтрации заключается в смещении маски фильтра по изображению с шагом один пиксель слева направо, сверху вниз.

При этом отклик задается суммой произведений коэффициентов фильтра на соответствующие значения пикселей в области, покрытой маской фильтра. Для маски размером 3×3 отклик r определяется следующим образом:

$$r = w_{(-1,-1)}f_{(i-1,j-1)} + w_{(-1,0)}f_{(i-1,j)} + w_{(-1,1)}f_{(i-1,j+1)} + \\ + w_{(0,-1)}f_{(i,j-1)} + w_{(0,0)}f_{(i,j)} + w_{(0,1)}f_{(i,j+1)} + w_{(1,-1)}f_{(i+1,j-1)} + \\ + w_{(1,0)}f_{(i+1,j)} + w_{(1,1)}f_{(i+1,j+1)}.$$

В общем случае фильтрация изображения f_{ij} размером $M \times N$ с использованием маски размером $m \times n$ описывается выражением

$$g_{ij} = \sum_{s=-a}^a \sum_{t=-b}^b w_{st} f_{i+s,j+t},$$

где $a = (m-1)/2$, $b = (n-1)/2$.

Таким образом, можно использовать маски размером не только 3×3 , но и более, например, 5×5 , 7×7 и т. п.

Если центр маски находится на границе изображения, то ее края выходят за его пределы. Для обработки краев изображений используются следующие приемы: дополнение средним значением, доопределение повторением крайних отсчетов последовательности, экстраполяция более высокого порядка.

Усредняющая или сглаживающая пространственная фильтрация может служить эффективным средством подавления высокочастотных шумов. Сглаживающие маски, часто называемые шумоподавляющими, имеют вид:

$$H1 = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}, \quad H2 = \frac{1}{10} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 1 \end{bmatrix}, \quad H3 = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}.$$

Повышение резкости изображений может быть достигнуто путем численного дифференцирования. С принципиальной точки зрения величина отклика оператора производной в точке изображения пропорциональна степени разрыва изображения в данной точке. Таким образом, дифференцирование изображения позволяет усилить перепады яркости и другие разрывы (шумы, помехи) и не подчеркивать области с медленными изменениями яркостей.

Для вычисления двумерной второй производной и наложения результата на изображение (высокочастотная фильтрация) используются маски:

$$H4 = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix} \quad H5 = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 9 & -1 \\ -1 & -1 & -1 \end{bmatrix} \quad H6 = \begin{bmatrix} 1 & -2 & 1 \\ -2 & 5 & -2 \\ 1 & -2 & 1 \end{bmatrix}.$$

Следует отметить, что для сохранения постоянного уровня яркости изображения коэффициент передачи маски, определяемый как сумма всех элементов маски, должен быть равен единице. Это справедливо для представленных масок H1–H6.

Для выделения контуров изображений можно использовать маски H1–H6 с уменьшенным на единицу центральным элементом.

В настоящее время рассмотренные алгоритмы и множество других, применяемых для решения задач обработки изображений и компьютерного зрения, реализованы в различных библиотеках компьютерного зрения, среди которых можно выделить OpenCV [1], dlib [2], PCL, VXL, AForge.Net, LTI и др.

Библиотека компьютерного зрения OpenCV

OpenCV (Open Source Computer Vision, <https://opencv.org/>) [1] – библиотека компьютерного зрения с открытым исходным кодом и широким спектром возможностей. Функциональность OpenCV доступна на языках C, C++, Python, Java, Ruby. Поддерживаются основные операционные системы: MS Windows, Linux, Mac, Android, iOS.

Библиотека включает большое количество оптимизированных классических и современных алгоритмов обработки изображений, распознавания образов, компьютерного зрения и машинного обучения, которые могут быть использованы для обнаружения и распознавания лиц, идентификации разных классов объектов, анализа действий человека в видео, отслеживания движений камеры, сопровождения движущихся объектов на видео и других.

OpenCV имеет модульную структуру, а это значит, что пакет включает в себя несколько общих или статических библиотек. Библиотека непрерывно обновляется, в версии OpenCV 4.5.5 доступны:

1) модуль *core* определяет основные структуры данных, используемые всеми остальными модулями, такие как:

- операции с массивами;
- сохранение XML/YAML/JSON;

- кластеризация;
 - утилиты, системные функции и макросы;
 - совместимость с OpenGL;
 - алгоритмы оптимизации;
 - параллельная обработка;
 - аппаратное ускорение;
- 2) модуль *imgproc* предназначен для обработки изображений и включает:
- фильтрацию изображений;
 - геометрические преобразования;
 - функции рисования;
 - функции преобразования цветного пространства;
 - анализ гистограмм;
 - обнаружение признаков;
 - сегментацию;
 - обнаружение объектов;
- 3) модуль *imgcodecs* используется для чтения и записи изображений;
- 4) модуль *videoio* предназначен для чтения и записи видео или последовательности изображений;
- 5) модуль *highgui* – графический интерфейс высокого уровня. Позволяет создавать и управлять окнами, которые могут отображать изображения и «запоминать» их содержимое;
- 6) модуль *video*, предназначенный для анализа видео, включающий алгоритмы оценки движения, вычитания фона и отслеживания объектов;
- 7) модуль *calib3d* – калибровка камеры и 3D-реконструкция. Включает основные алгоритмы геометрии с несколькими видами, калибровку одиночной и стереокамеры, оценку позы объекта, алгоритмы стереосоответствия и элементы 3D-реконструкции;
- 8) модуль *features2d* включает детекторы признаков, дескрипторы и средства сопоставления дескрипторов:
- обнаружение и описание признаков;
 - сопоставление дескрипторов;
 - рисование ключевых точек и совпадений;
 - классификация объектов;
- 9) модуль *objdetect*, предназначен для обнаружения объектов и экземпляров predetermined классов (например: лица, глаза, люди, автомобили и т. д.);

10) модуль *dnn*, предназначен для работы с глубокими нейронными сетями:

- инструментарий для загрузки моделей сетей из разных фреймворков;
- реализации готовых слоев нейронных сетей;
- утилиты, для создания новых слоев;

11) модуль *ml* – модуль для машинного обучения. Представляет собой набор классов и функций для статической классификации, регрессии и кластеризации данных;

12) модуль *flann* – модуль, содержащий коллекцию алгоритмов, оптимизированных для быстрого поиска ближайших соседей в больших наборах данных;

13) модуль *photo* – модуль для обработки фотографий:

- шумоподавление;
- HDR-визуализация (High Dynamic Range Rendering);
- обесцвечивание изображений с сохранением контраста;
- бесшовное клонирование, которое позволяет скопировать объект с одного изображения на другое таким образом, чтобы комбинация выглядела бесшовной и естественной;

14) модуль *stitching* используется для сшивания изображений:

- поиск признаков и сопоставление изображений;
- оценка вращения;
- автокалибровка (пытается оценить фокусные расстояния по заданной гомографии исходя из предположения, что камера вращается только вокруг своего центра, и может использоваться для построения панорамных мозаик);
- деформация изображений;
- оценка швов склеенных изображений;
- компенсация экспозиции;

15) модуль *gapi* – модуль, предназначенный для того, чтобы сделать обычную обработку изображений быстрой и переносимой путем внедрения новой графовой модели, суть которой заключается в построении графа на основе используемых операций к объектам данных. Графы представляются неявной структурой, для описания которой используются не «узлы» и «ребра», а операции и объекты данных. Объекты данных при этом

содержат не фактические значения, а описывают зависимости. Модуль выступает в качестве фреймворка.

Opencv-contrib – отдельный репозиторий, содержащий новые расширяющие функциональность модули, которые еще не включены в основную часть. Это связано с тем, что новые модули часто не имеют стабильного интерфейса и плохо протестированы, поэтому их и не включают в официальный дистрибутив OpenCV. *Opencv-contrib 4.5.5* содержит ряд модулей, среди них можно выделить основные:

- *alphamat* (альфа-матирование) предназначен для отделения объекта на переднем плане от фона обеспечивая нерезкие границы. Его можно использовать для дальнейших операций;

- *aruco* предоставляет возможности обнаружения двоичных квадратных реперных маркеров и инструменты для их использования при оценке позы человека и калибровке камеры;

- *barcode* служит для обнаружения и декодирования штрих-кодов;

- *bgsegm* применяется для вычитания фона при обработке видео;

- *bioinspired* позволяет обрабатывать как изображения, вызывающие оптические иллюзии, так и обычные;

- *ccalib* предназначен для калибровки одной камеры, стереопары и многокамерных систем. Позволяет удалять большие искажения на изображениях, а также реконструировать 3D-сцены на основе двух стереоизображений;

- *cvv* может быть использован для отладки приложений с использованием визуального пользовательского интерфейса;

- *dnn_objdetect* служит для обнаружения объектов с использованием глубоких сверточных нейронных сетей;

- *dnn_superres* позволяет обрабатывать изображения сверхвысокого разрешения, также включает функции масштабирования фото и видео;

- *face* предназначен для обнаружения и распознавания лиц на изображениях;

- *fuzzy*. Модуль реализует алгоритмы обработки изображений, основанные на нечеткой математике;

- *mcc* применяется для цветокоррекции с использованием Color correction Model;

- *tracking* используется для сопровождения различных объектов на видео;

- *ximgproc* предназначен для фильтрации несоответствий на стерео-изображениях, может быть использован для быстрого обнаружения границ с использованием алгоритма Structured Forest;
- *xphoto* применяется для восстановления поврежденных или отсутствующих частей изображения [3].

Библиотека компьютерного зрения dlib

Библиотека *dlib* включает реализацию множества традиционных математических функций, операций линейной алгебры, а также алгоритмов сжатия данных, классической обработки изображений и алгоритмов на основе машинного обучения, реализованных в основном на языке C++, однако ряд инструментов *dlib* возможно использовать и на Python.

Лицензирование *dlib* с открытым исходным кодом позволяет использовать ее бесплатно [2]. Следует выделить следующие основные инструменты в *dlib* версии 19.23:

1) *для обработки изображений* – функции для работы с различными типами пикселей; чтение и запись различных форматов изображений, таких как BMP, PNG, JPEG, GIF и формат *dlib* файла DNG; автоматическое преобразование цветового пространства; алгоритмы поиска объектов на изображениях; традиционные операции с изображениями, такие как поиск краев и морфологические операции и др.; алгоритмы извлечения признаков SURF (Speeded Up Robust Features, Ускоренные надежные признаки), HOG (Histogram of Oriented Gradients, Гистограмма направленных градиентов) и FHOOG (Felzenszwalb Histogram of Oriented Gradients, Гистограмма направленных градиентов Фельзеншвальба);

2) *для машинного обучения* – алгоритмы бинарной и многоклассовой классификации, регрессионного и структурного анализа, кластеризации, глубокого обучения, обучения без учителя, обучения с подкреплением;

3) *для метапрограммирования* – предоставляются функциональные возможности для отладки, выполнения различных операций с шаблонами. Включает большое число макрофункций;

4) *для вычислений*:

- быстрые алгоритмы обработки матричных объектов;
- многочисленные функции линейной алгебры и математические операции;

5) *для оптимизации* *dlib* включает нелинейные оптимизаторы общего назначения;

6) для работы с сетью в dlib имеется набор объектов для создания многопоточных программ, для обеспечения соединения между сетевыми приложениями, для создания сервера и т. д.

7) для работы с текстом имеются инструменты для анализа и различных манипуляций: преобразования кодировок, работы со строками, анализа параметров аргументов командной строки и др.

Практическая часть

При необходимости установить OpenCV и dlib. В приложении 1.1 показана последовательность установки для OpenCV4.5.5 и dlib 19.23.

Задание 1. Загрузить полноцветное изображение с использованием OpenCV и выполнить преобразования согласно своему варианту. Исследовать влияние параметров функций, необходимых для выполнения преобразования. Описание функций OpenCV4.5.5, необходимых для выполнения данной работы представлено в приложении 1.2. Подписать на изображении выполненное преобразование и свою фамилию.

Реализовать следующие операции обработки изображений:

№ Варианта	Описание
Вариант 1	Кадрирование, перевод изображения в полутоновое (в градациях серого).
Вариант 2	Поворот, перевод изображения в бинарное.
Вариант 3	Масштабирование, размытие изображения.
Вариант 4	Кадрирование, увеличение четкости.
Вариант 5	Поворот, линейное улучшение контраста.
Вариант 6	Масштабирование, улучшение контраста на основе эквализации гистограммы (clahe).
Вариант 7	Кадрирование, выделение контуров.
Вариант 8	Поворот, эффект рельефа.
Вариант 9	Масштабирование, увеличение четкости.
Вариант 10	Кадрирование, линейное улучшение контраста.
Вариант 11	Поворот, улучшение контраста на основе эквализации гистограммы (clahe).
Вариант 12	Масштабирование, размытие изображения.
Вариант 13	Кадрирование, эффект рельефа.

Вариант 14	Поворот, выделение контуров.
Вариант 15	Масштабирование, адаптивная бинаризация.

Задание 2. Загрузить полноцветное изображение лица человека с использованием OpenCV. Применить функцию выделения особых точек лица библиотеки dlib для изображений одного и тоже человека, полученных в различных условиях съемки. Сделать выводы о корректности нахождения точек лица на изображениях.

Перечень контрольных вопросов

- 1- Что понимают под цифровым изображением?
- 2- Сформулируйте процедуру преобразования цветного изображения в полутоновое (полутонового в бинарное).
- 3- Какие утилиты используются для установки OpenCV и dlib?
- 4- Перечислите основные модули, входящие в состав OpenCV.
- 5- Какими основными функциональными возможностями обладает библиотека dlib?

Установка библиотек компьютерного зрения в ОС Windows

Для установки OpenCV в Windows необходимо с официального сайта <https://opencv.org/releases/> скачать версию OpenCV для Windows (рисунок 1.4).

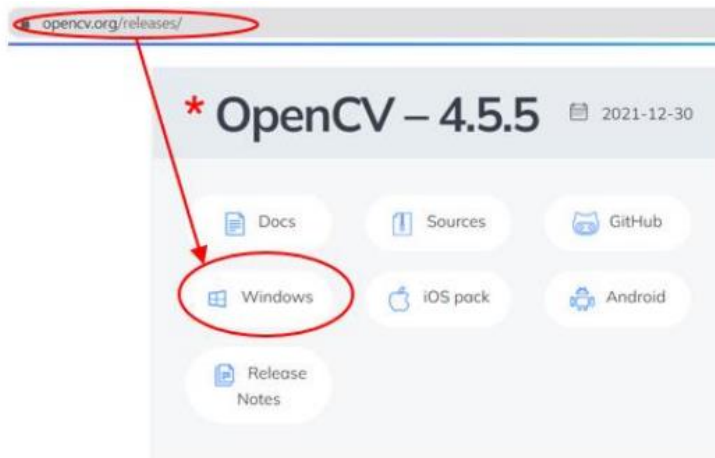


Рисунок 1.4. – Расположение OpenCV 4.5.5 на официальном ресурсе

Распаковывать архив в C:\opencv_4.5.5\build и установить значение системной переменной, как показано на рисунке 1.5.

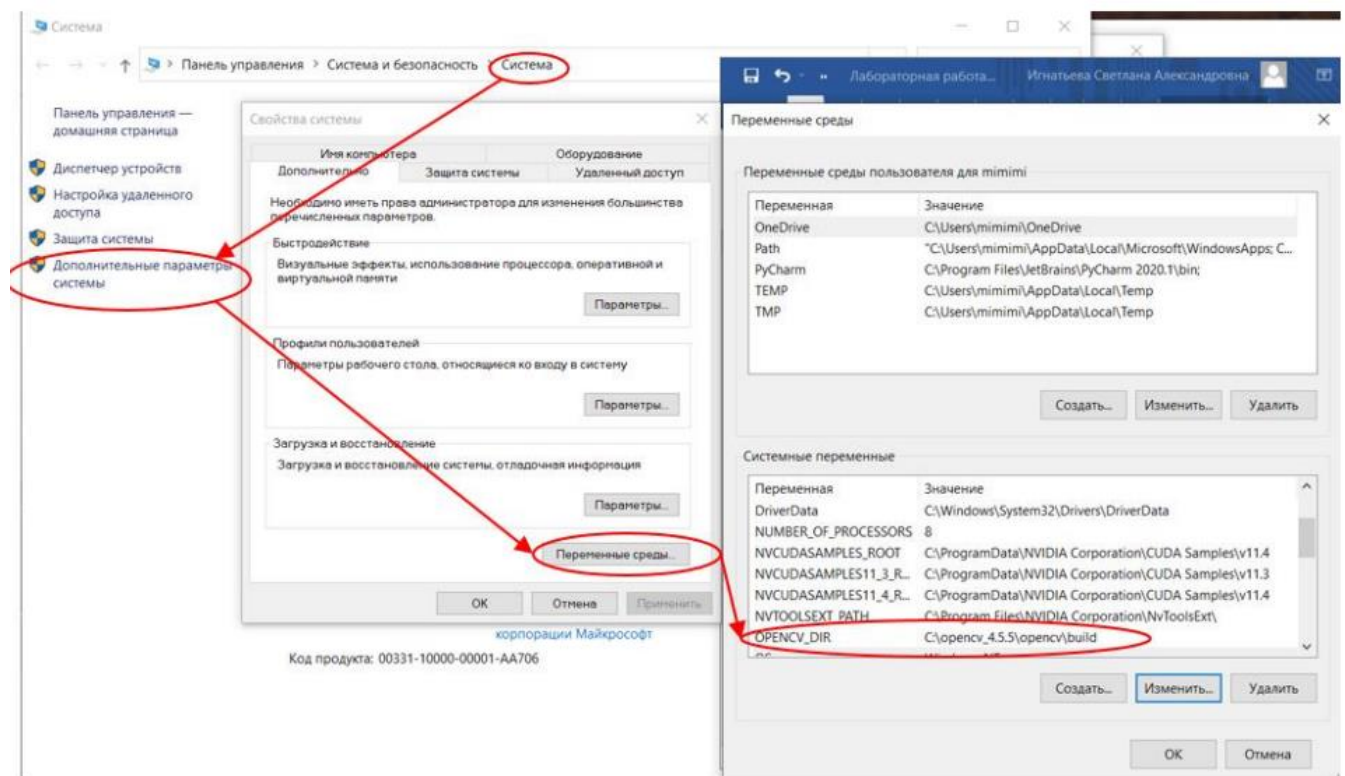


Рисунок 1.5. – Установка OpenCV 4.5.5 на персональный компьютер

Установка OpenCV в среду Python: `pip install opencv-python`

Установка OpenCV-contrib в среду Python: `pip install opencv-contrib-python`

Для установки библиотеки dlib необходимо использовать кроссплатформенную утилиту CMake, которая обладает возможностями автоматизации сборки программного обеспечения из исходного кода. Для

этого необходимо загрузить установочный файл CMake, доступный по ссылке <https://cmake.org/download/> (рисунок 1.6).

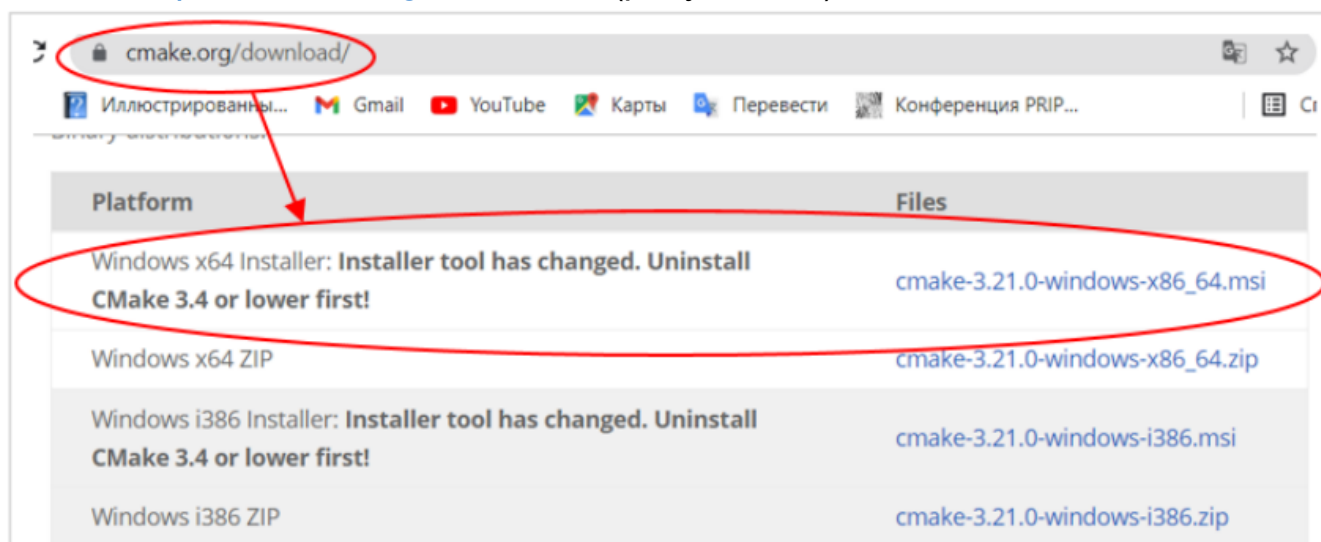


Рисунок 1.6. – Доступ к CMake на веб-ресурсе

При установке рекомендуется выбрать опцию «Add CMake to the system PATH for all users» («Добавить пути в переменные среды для всех пользователей»).

Dlib разработана на основе языка программирования C, следовательно, необходим компилятор. Автономный компилятор MS Visual Studio можно скачать по ссылке: <https://visualstudio.microsoft.com/ru/visual-cpp-build-tools/>. После завершения установки необходимо установить дополнительные пакеты для программирования C/C++, т. е. Packages CMake tools для Windows.

Чтобы установить dlib в среду Python требуется установить cmake: `pip install cmake`, а затем dlib: `pip install dlib`.

Приложение 1.2

Функции библиотеки OpenCV

`cv2.imread()` – чтение изображений в Python с помощью OpenCV. Возвращает 2D- или 3D-матрицу в зависимости от количества цветных каналов, присутствующих на изображении.

Синтаксис: `cv2.imread('Path/filename', flag)`.

`flag` является необязательным и может принимать одно из следующих значений: `cv2.imread_color` (считывает изображение с цветами RGB, но без канала прозрачности. Это значение по умолчанию для флага, когда значение не указано в качестве второго аргумента для `cv2.imread()`);

`cv2.imread_grayscale` (читает изображение как серое. Если исходное изображение является цветным, значение серого для каждого пикселя вычисляется путем получения среднего значения цветовых каналов и считывается в массив);
`cv2.imread_unchanged` (читает изображение как есть из источника).

`cv2.imwrite ()` – запись изображения в Python с помощью OpenCV.

Синтаксис: `cv2.imwrite ('Path/filename', image)`

`cv.imshow()` – вывод изображения на экран.

Синтаксис: `cv.imshow("filename", image)`

`cv.waitKey()` – задержка отображения окна в течении заданных миллисекунд, или до нажатия какой-либо клавиши на клавиатуре. Если в качестве параметра принято число, то время ожидания до закрытия окна ограничено указанным числом в миллисекундах. Если указан 0 или параметр не указан, окно закроется по нажатию любой клавиши клавиатуры.

`cv.resize` – изменение размера изображения.

Синтаксис:

`cv.resize(image, new_size, interpolation=cv2.INTER_XXX)`

`interpolation=cv2.INTER_XXX` – алгоритм интерполяции, может принимать значения: `cv2.INTER_NEAREST` (интерполяция ближайшего соседа); `cv2.INTER_LINEAR` (билинейная интерполяция); `cv2.INTER_CUBIC` (бикубическая интерполяция); `cv2.INTER_AREA` (пересчет с использованием отношения площадей пикселей); `cv2.INTER_LANCZOS4` (интерполяция Lanczos по окрестности 8×8).

`cv2.flip()` – переворачивает 2D-массив вокруг вертикальной, горизонтальной или обеих осей. Может принимать значения: 1 – переворот вокруг оси y, 0 – переворот вокруг оси z, -1 – переворот по обеим осям.

`cv2.split()` – разделить изображение по каналам.

Синтаксис:

`b, g, r = cv2.split(image)`

`cv2.merge()` – объединить каналы изображения.

Синтаксис:

`cv2.split([b, g, r])`

`cv2.blur()` – размытие изображения на основе свертки с ядром $n \times n$. Пиксель в центре матрицы устанавливается равным среднему значению всех остальных пикселей. Чем больше ядро свертки (т. е. n), тем больше размытие.

Синтаксис:

`cv2.blur(image, (n, n))`

`cv2.GaussianBlur()` – размытие по Гауссу. Похоже на предыдущее размытие, однако вместо простого среднего используется взвешенное, т. е.

чем ближе пиксели к центральному, тем большее влияние они оказывают на среднее значение.

Синтаксис:

`cv2.GaussianBlur(image, (n, n), s)`, где n – ядро свертки, s – стандартное отклонение ядра Гаусса. Установив значение 0, оно будет вычисляться автоматически.

`cv2.putText()` – наложение текста на изображение.

Синтаксис:

`cv2.putText(image, text, org, font, fontScale, color[, thickness[, lineType]])`, где `image` – изображение; `text` – строка текста; `org` – координаты нижнего левого угла текстовой строки x и y ; `font` – шрифт, например `FONT_HERSHEY_SIMPLEX`, `FONT_HERSHEY_PLAIN` и т. д.; `fontScale` – масштабный коэффициент шрифта, который умножается на базовый размер; `color` – цвет строки, `thickness` – толщина; `lineType` – необязательный параметр, который определяет тип линии.

`cv2.line()` – начертить линию на изображении.

Синтаксис:

`cv2.line(image, org, color[, thickness[, lineType[, shift]])`, где `image` – изображение; `text` – строка текста; `org` – координаты; `color` – цвет; `thickness` – толщина; `lineType` – необязательный параметр, который определяет тип линии; `shift` – цвет.

`cv2.rectangle()` начертить прямоугольник на изображении.

Синтаксис: `cv2.line(image, org1, org2 color[, thickness[, lineType[, shift]])`, где `image` – изображение; `text` – строка текста; `org1` – координаты верхнего левого угла; `org2` – координаты нижнего правого угла; `color` – цвет; `thickness` – толщина; `lineType` – необязательный параметр, который определяет тип линии; `shift` – цвет.

`cv2.cvtColor()` – преобразование из одного цветового пространства в другое.

Синтаксис: `cv2.cvtColor(image, code[, outImg[, CoutImg]])`, где `image` – изображение, `code` – код цветового пространства; `outImg` – необязательный параметр, выходное изображение того же размера и глубины, что и исходное; `CoutImg` – количество каналов в целевом изображении. Если равен 0, то количество каналов определяется автоматически, на основе исходного изображения. Является необязательным параметром.

`cv2.threshold()` – функция бинаризации. Для каждого пикселя применяется одно и то же пороговое значение. Если значение пикселя меньше

порогового значения, оно устанавливается равным 0, в противном случае оно устанавливается на максимальное значение.

Синтаксис: `cv2.threshold (image, thr, max, TypeThr)`, где `image` – исходное изображение, которое должно быть изображением в градациях серого; `thr` – это пороговое значение, которое используется для классификации значений пикселей; `max` – это максимальное значение, которое присваивается значениям пикселей, превышающим пороговое значение. OpenCV предоставляет различные типы пороговых значений, которые задаются четвертым параметром функции: `cv2.THRESH_BINARY`, `cv2.THRESH_BINARY_INV`, `cv2.THRESH_TRUNC`, `cv2.THRESH_TOZERO`, `cv2.THRESH_TOZERO_INV`.

`cv2.adaptiveThreshold()` – функция адаптивной бинаризации, устанавливает порог для пикселя на основе небольшой области вокруг него. Таким образом, для разных областей изображения используется разный порог.

Синтаксис: `cv2.adaptiveThreshold(image, maxVal, adaptiveMethod, TypeThr, blockSize, const)`, где `image` – исходное одноканальное изображение; `maxVal` – максимальное значение, которое может быть присвоено; `adaptiveMethod` – адаптивный метод, который определяет как рассчитывается пороговое значение. Может принимать значение `cv2.ADAPTIVE_THRESH_MEAN_C()`, `cv2.ADAPTIVE_THRESH_GAUSSIAN_C()`.

`cv2.findContours()` – функция выделения контуров на изображении.

Синтаксис: `cv2.findContours(image, mode, method)`, где `image` – исходное изображение; `mode` – один из четырех режимов группировки найденных контуров: `CV_RETR_LIST` – выдаёт все контуры без группировки; `CV_RETR_EXTERNAL` – выдаёт только крайние внешние контуры; `CV_RETR_CCOMP` – группирует контуры в двухуровневую иерархию. На верхнем уровне – внешние контуры объекта. На втором уровне – контуры отверстий, если таковые имеются. Все остальные контуры попадают на верхний уровень; `CV_RETR_TREE` – группирует контуры в многоуровневую иерархию; `method` – один из трёх методов упаковки контуров: `CV_CHAIN_APPROX_NONE` – упаковка отсутствует и все контуры хранятся в виде отрезков, состоящих из двух пикселей; `CV_CHAIN_APPROX_SIMPLE` – склеивает все горизонтальные, вертикальные и диагональные контуры; `CV_CHAIN_APPROX_TC89_L1`, `CV_CHAIN_APPROX_TC89_KCOS` – применяет к контурам метод упаковки (аппроксимации) Teh-Chin.

`cv2.filter2D()` – может применяться для сглаживания или увеличения резкости изображений в зависимости от ядра свертки и выполнять другие преобразования изображений.

Синтаксис: `cv2.Filter2D(src, ddepth, kernel)`, где `src` – исходное изображение; `ddepth` – глубина, значение `-1` означает, что результирующее изображение будет иметь ту же глубину, что и исходное; `kernel` – матрица фильтра, применяемая к изображению.

Для повышения резкости может использоваться одна из масок НЗ–Н6, для получения эффекта рельефа – ядро

$$kernel = \begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}.$$

`cv2.convertTo()` – выполняет преобразование изображения для изменения контраста и яркости $new_img = \alpha \cdot img + \beta$, где `new_img` – преобразованное изображение, α и β – параметры, для управления яркостью и контрастом.

Синтаксис: `img.convertTo(new_img, ddepth, alfa, beta)`, где `img` – исходное изображение; `new_img` – изображение с преобразованием; `ddepth` – глубина; `alfa` и `beta` – параметры для управления яркостью и контрастом.

`cv2.createCLAHE()` – алгоритм адаптивной коррекции гистограмм с ограниченным контрастом. Изображение разделяется на небольшие блоки, и гистограмма выравнивается для каждого из них, после чего для объединения каждого блока применяется билинейная интерполяция для устранения искусственных границ. Применяется для улучшения контрастности изображений. Имеет два параметра: `clipLimit` (устанавливает порог для ограничения контраста, по умолчанию 40) и `tileGridSize` (устанавливает количество плиток в строке и в столбце, по умолчанию 8×8).

Функции библиотеки dlib

Для выполнения лабораторной работы необходимы следующие инструменты из библиотеки `dlib`:

– `get_frontal_face_detector()` – функция для детектирования лиц. Для обнаружения лиц `dlib` использует методы HOG (Histogram of Oriented Gradients) и SVM (Support Vector Mashines) [5].

– `shape_predictor()` – представляет собой инструмент, который выбирает область изображения, содержащую некоторый объект, и выводит набор местоположений точек, определяющих положение объекта. В качестве аргумента необходимо указать используемую обученную модель `shape_predictor_68_face_landmarks.dat`.

Пример 68-точечной модели определения особых точек лица в `dlib` показан на рисунке 1.7.



**Рисунок 1.7. – Расположение
ключевых точек лица
в модели dlib**

Примеры реализации базовых процедур обработки изображений на языке программирования Python с использованием OpenCV

Преобразование цветного изображения в градации серого

```
import os, cv2 as cv
image = cv.imread("./img2.jpg")
cv.imshow("original", image)
gray_image = cv.cvtColor(image, cv.COLOR_BGR2GRAY)
cv.imshow("gray_image", gray_image)
cv.waitKey()
```

Добавление тестовой надписи на изображение

```
import os, cv2 as cv
image = cv.imread("./img2.jpg")
output = image.copy()
cv.putText(output, "Image with text", (50, 50),
           cv.FONT_HERSHEY_SIMPLEX, 2, (250, 80, 50), 3)
cv.imshow("Изображение с текстом", output)
cv.waitKey()
```

*Пример фрагмента кода python для детектирования
особых точек лица с использованием библиотеки dlib*

Выбор детекторов:

```
# детектор лица на изображении
detector = dlib.get_frontal_face_detector()
# детектор контура лица (68 точек)
predictor = dlib.shape_predictor('shape_predictor_68_face_landmarks.dat')
for file in face_images:
    image = cv2.imread(file)
```

Поиск и отображение на изображение особых точек лица:

```
rects = detector(gray, 1)
for rect in rects:
    # нахождение 68 точек для каждого найденного лица
    shape = predictor(gray, rect)
    for i in range(0, 68):
        # отображение точек
        cv2.circle(image, (shape.part(i).x, shape.part(i).y),
                       1, (0, 0, 255), -1)
cv2.imwrite(f'./output/{os.path.split(file)[-1]}', image)
```